Lab 5 Protocol: Microservices with Consul

Project

Repository branch: `micro_consul`

GitHub link: https://github.com/senivan/05-consul/tree/micro_consul

Implementation variant: Consul.

Services:

- `facade-service`
- `logging-service`
- `counter-service`

Infrastructure:

- Consul `1.20`
- RabbitMQ `3.13-management`
- Hazelcast `5.5`

How It Was Run

```
docker compose up --build -d
docker compose up -d --scale logging-service=2 --scale counter-service=2
```

Final scaled state:

```
05-consul-consul-1            Up, port 8500
05-consul-facade-service-1    Up, port 8000
05-consul-logging-service-1   Up
05-consul-logging-service-2   Up
05-consul-counter-service-1   Up
05-consul-counter-service-2   Up
05-consul-hazelcast-1         Up, port 5701
05-consul-rabbitmq-1          Up healthy, ports 5672 and 15672
```

Consul KV Configuration

The `consul-kv-seed` container successfully wrote:

```
config/hazelcast/cluster_name=dev
config/hazelcast/addresses=hazelcast:5701
config/rabbitmq/url=amqp://guest:guest@rabbitmq:5672/%2F
config/rabbitmq/queue=messages
```

Validation:

```
curl -sS 'http://localhost:8500/v1/kv/config/rabbitmq/queue?raw'
```

Result:

```
messages
```

Consul Service Registration

Validation:

```
curl -sS http://localhost:8500/v1/catalog/services
```

Result:

```
{"consul":[],"counter-service":[],"facade-service":[],"logging-service":[]}
```

Scaled health checks showed two passing instances for `logging-service` and two passing instances for `counter-service`.

Example `logging-service` instances:

```
logging-service-172.20.0.6-8001 passing
logging-service-172.20.0.8-8001 passing
```

Example `counter-service` instances:

```
counter-service-172.20.0.7-8002 passing
counter-service-172.20.0.9-8002 passing
```

API Validation

Health:

```
curl -sS http://localhost:8000/health
```

Result:

```
{"status":"ok"}
```

POST message:

```
curl -sS -X POST http://localhost:8000/messages \
  -H 'content-type: application/json' \
  -d '{"message":"hello consul integration test"}'
```

Result:

```
{
  "stored": {
    "id": "95e9fdc7-d631-4417-96fe-6dfeaadd5cf3",
    "message": "hello consul integration test"
  },
  "logging_service": "http://172.20.0.5:8001"
}
```


After adding timing instrumentation, POST responses also include contribution timing:


```
{
  "stored": {
    "id": "303a8a89-1c69-4fdf-aa54-6e11d74cdb7e",
    "message": "timing check"
  },
  "logging_service": "http://172.20.0.6:8001",
  "timings_ms": {
    "total": 43.029,
    "logging_service": 8.822,
    "counter_service": 23.923
  }
}
```


GET messages:

```
curl -sS http://localhost:8000/messages
```

Result: returned stored messages from Hazelcast through `logging-service`. After all validation and performance runs, the message count was `2423`.

GET counter:

```
curl -sS http://localhost:8000/counter
```

Result:


```
{"counter":551,"counter_service":"http://172.20.0.7:8002"}
```


Note: with two `counter-service` instances, RabbitMQ distributes messages between consumers. The counter value above is the local value from the discovered counter instance, not a global aggregate.

Failure / Failover Validation

One `logging-service` replica was force-killed:

```
docker kill 05-consul-logging-service-1
```

Consul health changed for that service instance:

```
logging-service-172.20.0.5-8001 critical
Output: Get "http://172.20.0.5:8001/health": context deadline exceeded
```

The remaining replica stayed passing:

```
logging-service-172.20.0.8-8001 passing
Output: HTTP GET http://172.20.0.8:8001/health: 200 OK
```

A facade call after the kill still succeeded and was routed to the passing logging instance:

```
curl -sS -X POST http://localhost:8000/messages \
  -H 'content-type: application/json' \
  -d '{"message":"after logging replica kill"}'
```

Result:

```
{
  "stored": {
    "id": "667a1198-4193-4ae0-8336-3e081ef256bb",
    "message": "after logging replica kill"
  },
  "logging_service": "http://172.20.0.8:8001"
}
```

Console Evidence

`facade-service` logs showed service discovery through Consul and RabbitMQ publishing:

```
GET http://consul:8500/v1/health/service/logging-service?passing=true "HTTP/1.1 200 OK"
POST http://172.20.0.8:8001/logs "HTTP/1.1 200 OK"
GET http://consul:8500/v1/kv/config/rabbitmq/url?raw=true "HTTP/1.1 200 OK"
published counter event to queue=messages
```

`logging-service` logs showed Consul registration, Consul KV reads, Hazelcast connection, and message writes:

```
registered logging-service as logging-service-<container-ip>-8001 in Consul
GET http://consul:8500/v1/kv/config/hazelcast/cluster_name?raw=true "HTTP/1.1 200 OK"
connected to Hazelcast cluster=dev members=['hazelcast:5701']
stored message id=...
```

`counter-service` logs showed Consul registration, Consul KV reads, RabbitMQ connection, and consumed messages:

```
registered counter-service as counter-service-172.20.0.7-8002 in Consul
GET http://consul:8500/v1/kv/config/rabbitmq/queue?raw=true "HTTP/1.1 200 OK"
started RabbitMQ consumer queue=messages
counted message #551: account-0: message-99
```

Performance Results

Command:

```
.venv/bin/python perf/performance_test.py --requests-per-account 100
```

Measured on the local Docker Desktop environment with:

- 1 `facade-service`
- 2 `logging-service` replicas
- 2 `counter-service` replicas
- 100 requests per account

Test scenarios	Task 1 (in-mem)	Task 3 (DB)	Task 5 (final)
10 accounts	Total time: not available	Total time: not available	Total time: `33.776s`
	logging-service contribution: not available	logging-service contribution: not available	logging-service contribution: `6.588s`
	counter-service contribution: not available	counter-service contribution: not available	counter-service contribution: `18.396s`
1 account	Total time: not available	Total time: not available	Total time: `3.504s`
	logging-service contribution: not available	logging-service contribution: not available	logging-service contribution: `0.674s`
	counter-service contribution: not available	counter-service contribution: not available	counter-service contribution: `1.932s`

Raw output:

```
10 account(s): total=33.776s logging=6.588s counter=18.396s avg=0.0338s p95=0.0394s
1 account(s): total=3.504s logging=0.674s counter=1.932s avg=0.0350s p95=0.0428s
```

The counter contribution is measured as the facade request latency spent in the counter path: reading RabbitMQ configuration from Consul KV and publishing the message to RabbitMQ. Counter consumption is asynchronous, so it is not fully included in synchronous request latency.

Validation Commands Run

```
python3 -m compileall services scripts perf
docker compose up --build -d
docker compose up -d --scale logging-service=2 --scale counter-service=2
curl -sS http://localhost:8000/health
curl -sS -X POST http://localhost:8000/messages -H 'content-type: application/json' -d
'{"message":"hello consul integration test"}'
curl -sS http://localhost:8000/messages
curl -sS http://localhost:8000/counter
curl -sS http://localhost:8500/v1/catalog/services
curl -sS 'http://localhost:8500/v1/health/service/logging-service?passing=true'
curl -sS 'http://localhost:8500/v1/health/service/counter-service?passing=true'
docker kill 05-consul-logging-service-1
curl -sS 'http://localhost:8500/v1/health/service/logging-service'
.venv/bin/python perf/performance_test.py --requests-per-account 100
```

Screenshots To Attach

The implementation and command validation are complete. For final submission, attach screenshots of:

- Consul UI at `http://localhost:8500` showing `facade-service`, `logging-service`, and `counter-service`.
- Consul UI showing one critical `logging-service` instance after the kill test.
- `POST /messages` response.
- `GET /messages` response.
- `GET /counter` response.
- Logs for `facade-service`, `logging-service`, and `counter-service`.